

IF 45 I

Advance Web Programming

Meet2 - Modules & Packages in Node.js

Wirawan Istiono, S.Kom., M.Kom



Meet 2

Objective:
Sub-CLO07I2

Students are able to build basic web programming and feature in
Node JS (C6)

OUTLINE

- *Basic Code in NodeJS*
- *Built-in modules (fs, path, os).*
- *Creating custom modules (module.exports).*
- Installing packages with npm

Basic Code in NodeJS

Sample Plain Text NodeJS

```
const http = require("http");
```

```
const server = http.createServer((req, res) => {  
  res.writeHead(200, { "Content-Type": "text/plain" });  
  res.write("Sample Text Line1\n");  
  res.write("This is Sample Text Line2\n");  
  res.end();  
});
```

```
server.listen(3000, () => {  
  console.log("Server running at http://localhost:3000");  
});
```

[OPTIONAL]

This function is used to send response headers to the browser:

200 → means the status code is "OK" (request successful).

Content-Type → tells the browser what type of data to expect:

- "text/plain" → the browser will display plain text.
- "text/html" → the browser will render HTML.
- "application/json" → the browser/client knows it's JSON.

Sample HTML code NodeJS

```
const http = require("http");
```

```
const server = http.createServer((req, res) => {
```

```
  res.writeHead(200, { "Content-Type": "text/html" });
```

```
  res.write("<h1>Sample text line one for HTML</h1>");
```

```
  res.write("<i><u>This is sample line two for HTML</i></u>");
```

```
  res.end();
```

```
});
```

```
server.listen(3000, () => {
```

```
  console.log("Server running at http://localhost:3000");
```

```
});
```

After “**text/html**”

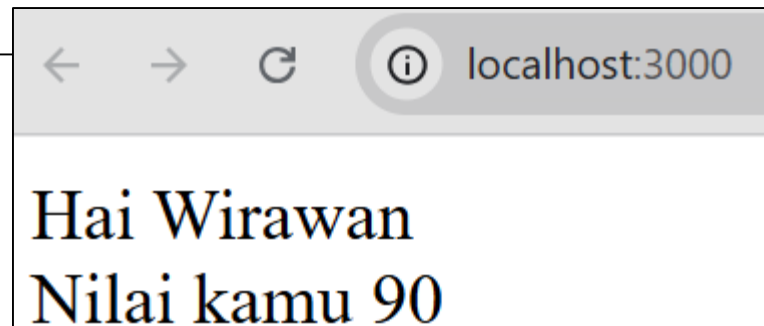
You can write html code like the sample beside

Variable Declaration

In Node.js, variables can be created with **const**, **let**, or **var**.

- Use **const** if the variable doesn't need to be reassigned.
- Use **let** if the variable needs to be changed.
- Avoid **var**, as it is considered obsolete and can cause bugs.

```
const server = http.createServer((req, res) => {  
  const nama = "Wirawan";  
  let nilai = 85;  
  
  res.writeHead(200, { "Content-Type": "text/html" });  
  
  res.write("<br>Hai " + nama);  
  nilai = 90;  
  res.write(`<br>Nilai kamu ${nilai} <br>`);  
  res.end();  
});
```



Print Variable

When you want to print or call variable, you can use two ways.

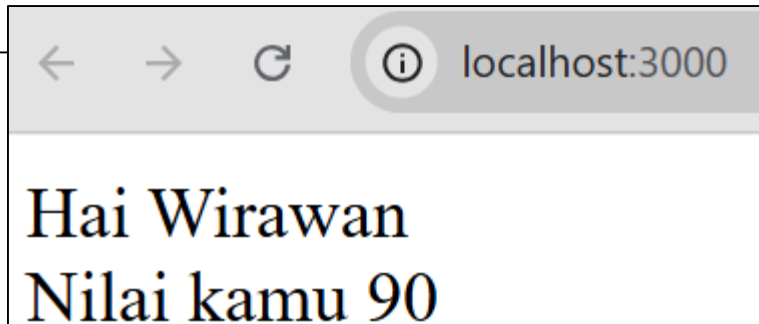
1. Using String concatenation quotation (") and Plus sign (+)

format: **"your text" + variable**

2. Using Template literals backtick (`) and call your variable in the bracket \${...}

Format: **`your text \${variable}`**

```
const server = http.createServer((req, res) => {  
  const nama = "Wirawan";  
  let nilai = 85;  
  
  res.writeHead(200, { "Content-Type": "text/html" });  
  
  res.write("<br>Hai " + nama);  
  nilai = 90;  
  res.write(`<br>Nilai kamu ${nilai} <br>`);  
  res.end();  
});
```



Basic Function / Method

There are 2 ways to use Function in NodeJS.

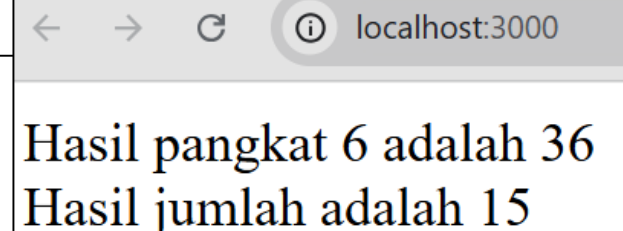
1. Function declaration

```
function funcName(param1, param_n) {  
    statement  
}
```

2. Function expression (const + arrow)

```
const funcName = (param1, param_n) => {  
    statement  
}
```

```
function hasil_pangkat(a) {  
    const hasil = a*a;  
    return hasil;  
}  
  
const hasil_jumlah = (a,b,res) => {  
    const jml = a+b;  
    res.write("<br>Hasil jumlah adalah " + jml);  
}  
  
const server = http.createServer((req, res) => {  
    const hasil = hasil_pangkat(6);  
    res.write("Hasil pangkat 6 adalah "+ hasil);  
    hasil_jumlah(5,10, res);  
    res.end();  
});
```



Built-in modules (fs, path, os)

Built-in modules (fs, path, os).

Node.js already provides many built-in modules that can be used directly without installing them via npm. Some of the most commonly used are fs (file system), path (file path utilities), and os (operating system).

- fs → manages files and folders (read, write, delete).
- path → safely manipulates file/directory paths.
- os → operating system information (CPU, RAM, user, etc.).

File System (FS)

The fs module is used to access and manage files or folders on the computer.

The File System (FS) can only be accessed locally on the server where Node.js is running.

This means:

- You can read/write files on your computer/server.
- Browsers cannot directly access the FS (for security reasons).

To access files from a browser (website), here's how:

- Create a Node.js server that uses the FS.
- The server reads local files.
- The results are sent to the browser via an HTTP response.

File System (FS)

```
const fs = require("fs");

// Menulis file baru
fs.writeFile("contoh.txt", "Halo, ini isi file dari Node.js!", (err) => {
  if (err) {
    console.error("Gagal menulis file:", err);
    return;
  }
  console.log("File berhasil dibuat!");
});
```

```
// Membaca isi file
fs.readFile("contoh.txt", "utf8", (err, data) => {
  if (err) {
    console.error("Gagal membaca file:", err);
    return;
  }
  console.log("Isi file adalah:");
  console.log(data);
});
```

After write and run the code above, please cek your folder file,
You'll found the txt files with filename **contoh.txt**



Path

- Paths only exist in Node.js (server-side).
- Browsers (client-side) don't have access to paths, as they involve the operating system's file structure.
- On a website/browser, we can only access the URL path, not the operating system's file path.

Example:

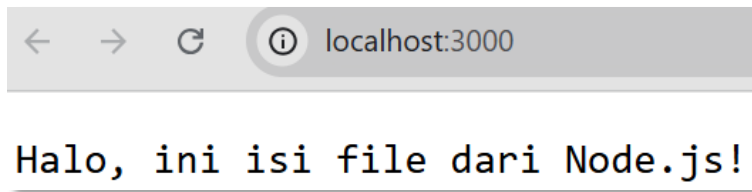
- `http://localhost:3000/folder/file.txt` ➞ This is the URL path.
- `C:\project\folder\file.txt` ➞ This is the path in the file system.

Paths in Node.js only work with the local/server file system, not directly on the website.

Sample Path

```
const http = require("http");
const fs = require("fs");
const path = require("path");

const server = http.createServer((req, res) => {
  const filePath = path.join(__dirname, "contoh.txt");
```



```
fs.readFile(filePath, "utf8", (err, data) => {
  if (err) {
    res.writeHead(500, { "Content-Type": "text/plain" });
    res.end("Error membaca file!");
    return;
  }

  res.writeHead(200, { "Content-Type": "text/plain" });
  res.end(data);
});

server.listen(3000, () => {
  console.log("Server berjalan di http://localhost:3000");
});
```

OS (Operating System)

OS is a built-in Node.js module for obtaining information about the operating system on which Node.js is running.

Its functions include:

- Determining the platform/OS (Windows, Linux, MacOS).
- Checking the CPU architecture (x64, arm, etc.).
- Viewing information about the CPU, memory, hostname, user, etc.
- Determining the user's home directory.
- Determining the number of cores for multi-threading optimization.

In other words, os provides information about the server environment, not about the browser or website.

Important note:

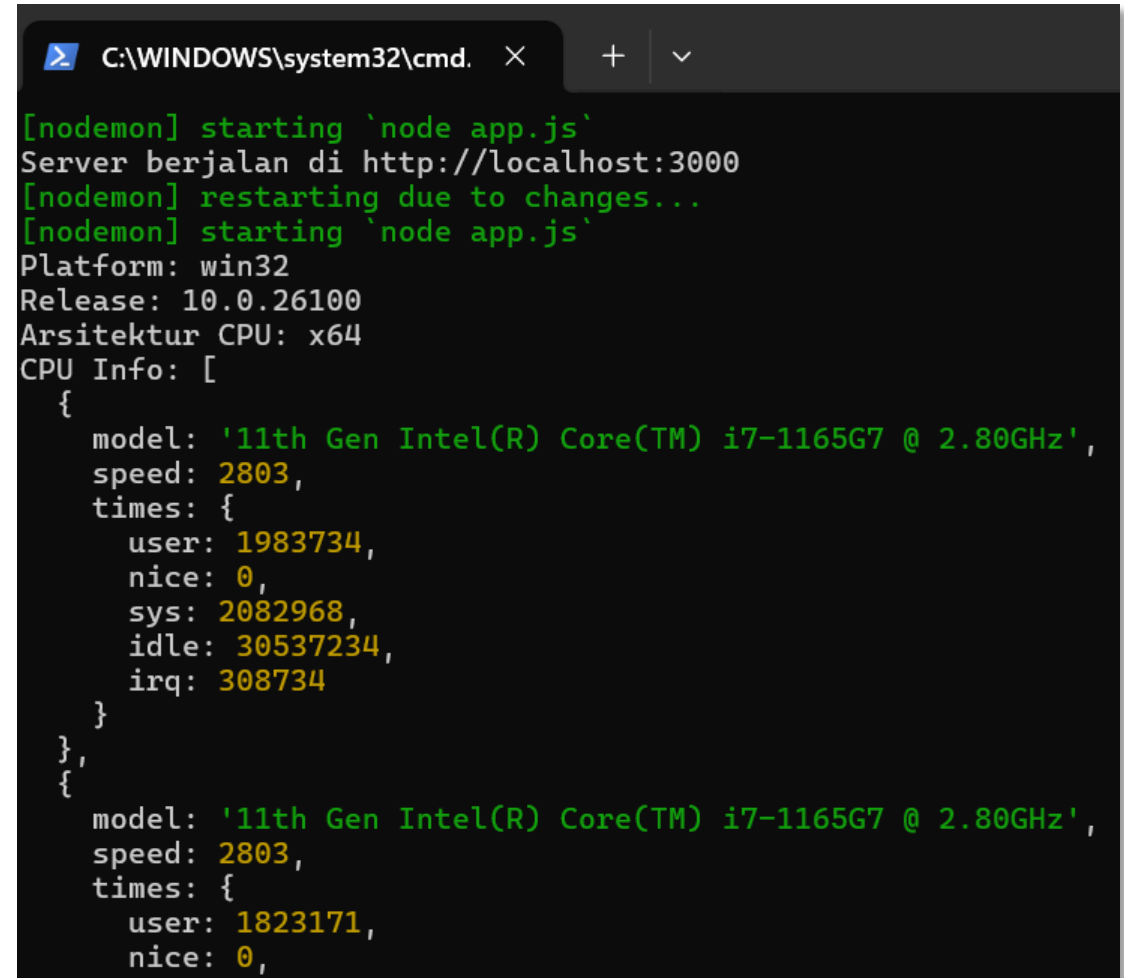
OS can only be used in Node.js (server-side), not in the browser.

It is generally used for server monitoring, application optimization, or creating CLI tools that require OS information.

OS sample

```
const os = require("os");

console.log("Platform:", os.platform());
console.log("Release:", os.release());
console.log("Arsitektur CPU:", os.arch());
console.log("CPU Info:", os.cpus());
console.log("Total Memori:", os.totalmem());
console.log("Free Memori:", os.freemem());
console.log("Home Directory:", os.homedir());
console.log("Hostname:", os.hostname());
console.log("Uptime (detik):", os.uptime());
```



```
C:\WINDOWS\system32\cmd. x + v

[nodemon] starting `node app.js`
Server berjalan di http://localhost:3000
[nodemon] restarting due to changes...
[nodemon] starting `node app.js`
Platform: win32
Release: 10.0.26100
Arsitektur CPU: x64
CPU Info: [
  {
    model: '11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz',
    speed: 2803,
    times: {
      user: 1983734,
      nice: 0,
      sys: 2082968,
      idle: 30537234,
      irq: 308734
    }
  },
  {
    model: '11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz',
    speed: 2803,
    times: {
      user: 1823171,
      nice: 0,
```

Creating custom modules (module.exports).

Custom Modules

- **module.exports**, used to export functions/variables/classes from a file so they can be used in other files.
- **require()**, used to import the module.

Create custom module

Create new file, with name **meet2_math_modules.js**, and write code below

```
function tambah(a, b) {  
  return a + b;  
}  
  
function kali(a, b) {  
  return a * b;  
}  
  
module.exports = {  
  tambah,  
  kali  
};
```

In **app.js** (or main files) files, write code below

```
const http = require("http");  
const mymath = require("./meet2_math_modules");  
  
const server = http.createServer((req, res) => {  
  res.writeHead(500, { "Content-Type": "text/html" });  
  res.write("Hasil kali: " + mymath.kali(5,7));  
  res.write("<br>Hasil tambah: " + mymath.tambah(32,27));  
  res.end();  
});  
  
server.listen(3000, () => {  
  console.log("Server berjalan di http://localhost:3000");  
});
```

Installing packages with npm

npm

npm = Node Package Manager

npm is Node.js's built-in package manager, used to:

- Install third-party packages (libraries)
- Manage dependencies in Node.js projects
- Create and publish your own packages

So, if Node.js is the "engine," then npm is the "app/library store" for Node.js.

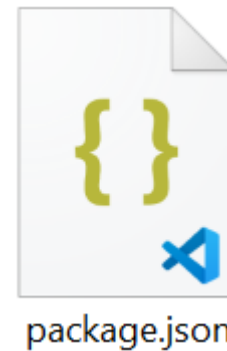
Note: Every Node.js project usually has a **package.json** file to record dependencies.

Create package json

- Make sure, your command prompt is in your project folder.
- To create package.json, write code below in commandprompt

npm init -y

```
{
  "name": "sample_project",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}
```



Lodash library

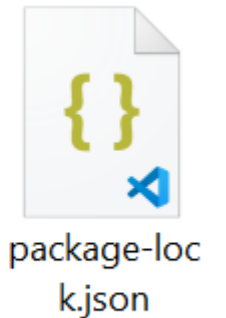
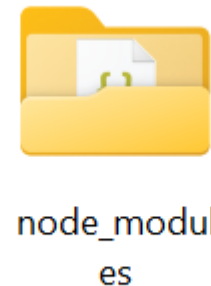
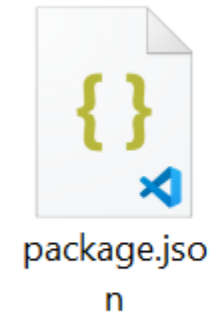
Lodash provides ready-to-use functions to simplify data manipulation, specifically Arrays, Objects, Strings, Numbers, and Functions.

With Lodash, we don't need to write lengthy basic functions, as they're already built-in.

- To install lodash, write this command to your commandprompt

npm install lodash

- Cek your package.json



Sample how to use lodash

In your app.js, write code below.

The code used to remove duplicate value in Array using loadsh library

```
const http = require("http");
const lodashlib = require("lodash");

const server = http.createServer((req, res) => {
  let arr = [1, 2, 2, 3, 4, 4, 5];

  res.writeHead(200, { "Content-Type": "text/html" });
  res.write("Hasil array: " + arr);
```

```
    //hilangkan angka duplicate pada array
    arr = lodashlib.uniq(arr);
    res.write("<br>Hasil array no duplicate " + arr);
    res.end();
  });

  server.listen(3000, () => {
    console.log("Server berjalan di http://localhost:3000");
  });
```

NEXT WEEK'S OUTLINE

- JSON Basics & CRUD with Files

REFERENCES

- B. Lawson and J. Encinas, Node.js Web Development, 6th ed., Packt Publishing, 2023
- R. Raj, Learning Node.js Development, Packt Publishing, 2018.
- E. Cantelon, M. Harter, T. Holowaychuk, and N. Rajlich, Node.js in Action, 2nd ed., Manning Publications, 2017.
- Maulana, a., bau, r.t.r., munawar, z., setiawan, r., aisa, s., istiono, w., irfan, a., pratama, y.a., ramadhany, e.d., pomalingo, s. And permana, a.a., 2023. *Pemrograman web 101: memahami dasar-dasar untuk mengembangkan situs web*. Get press indonesia.
- The Node.js Foundation, “Node.js Documentation,” [Online]. Available: <https://nodejs.org/en/docs/>. [Accessed: Jun. 5, 2025].
- Express.js Team, “Express.js Documentation,” [Online]. Available: <https://expressjs.com/>. [Accessed: Jun. 5, 2025].

Visi

Menjadi Program Studi Strata Satu Informatika **unggulan** yang menghasilkan lulusan **berwawasan internasional** yang **kompeten** di bidang Ilmu Komputer (*Computer Science*), **berjiwa wirausaha** dan **berbudi pekerti luhur**.



Misi

1. Menyelenggarakan pembelajaran dengan teknologi dan kurikulum terbaik serta didukung tenaga pengajar profesional.
2. Melaksanakan kegiatan penelitian di bidang Informatika untuk memajukan ilmu dan teknologi Informatika.
3. Melaksanakan kegiatan pengabdian kepada masyarakat berbasis ilmu dan teknologi Informatika dalam rangka mengamalkan ilmu dan teknologi Informatika.